Step 1 — A 2-Node Graph With No Branching, Just to Prove the Mechanics — Solution

Back to this step · Hint 1 · Hint 2 · Solution

Story — Step 1: every later step adds nodes and branches to one graph. Before any of that, prove the mechanics on the smallest graph that does something real: a typed state, two nodes, plain edges, and one model call. **If not:** when Step 3's branching misbehaves, you won't know if the problem is the branch or your understanding of how state flows.

After this step the project runs: `python -m documind.main` answers questions through a 2-node graph, and prints which nodes ran.

The shared files

These are the same files as Projects 1 and 2 — Doc01's config, errors and logger, and Project 1's prompt loader — with only the package name changed to `documind`.

Story — `exceptions.py`, `config.py`, `utils/logger.py`: settings load and fail loudly at startup; every file logs the same way. **If not:** a missing key would surface deep inside a graph node as a confusing error.

```
# src/documind/exceptions.py
# why: one file for every named error in the project, so any module can
# raise or catch a specific problem instead of a bare Exception.

# from Doc01
class MissingConfigError(Exception):
    """Raised when a required setting is missing from the environment."""
    pass
```

```
# src/documind/config.py
# how: os.getenv() is how we read env vars
import os
# why: @dataclass auto-builds __init__ for a typed holder
from dataclasses import dataclass
# how: reads .env and loads it into os.environ
from dotenv import load_dotenv
```

```
# why: a specific error type, not a bare Exception
from documind.exceptions import MissingConfigError

@dataclass
class Config:
    # why: a typed object, not a plain dict — your editor autocompletes
    # .openai_api_key and .log_level, and catches a typo'd field name.
    openai_api_key: str
    log_level: str

def require_env(key: str) -> str:
    # why: one place that turns a missing/empty env var into a clear,
    # named error, instead of repeating the same if-check for every key.
    # when: called once per required key, inside load_config().
    value = os.getenv(key)
    if value is None or value == "":
        # how: both "never set" (None) and "set but blank" (empty string)
        # count as missing — an empty API key is never usable either way.
        raise MissingConfigError(
            f"Required environment variable is missing: {key}"
        )
    return value

def load_config() -> Config:
    # why: the one function every other file calls to get settings — so a
    # missing key fails loudly here, at startup, not confusingly later.
    # when: call this once, at the top of main(), before anything else.
    # how: reads .env into os.environ; must run before os.getenv() below
    load_dotenv()
    # how: required — raises if missing
    api_key = require_env("OPENAI_API_KEY")
    # how: optional — "INFO" if unset
    log_level = os.getenv("LOG_LEVEL", "INFO")
    return Config(openai_api_key=api_key, log_level=log_level)
```

```
# src/documind/utils/logger.py
import logging
import os

def get_logger(name: str) -> logging.Logger:
    # why: gives every file in the project the same logging setup from one
    # place, instead of each file configuring handlers its own way.
    # when: logging.getLogger(name) always returns the same object for the
    # same name, so calling this again in the same run is safe and cheap.
    logger = logging.getLogger(name)
```

```
    if not logger.handlers:
        # how: only attach a handler the first time this name is
        # configured — without this guard, every line would print twice.
        level_name = os.getenv("LOG_LEVEL", "INFO")
        # how: falls back to INFO if the env value is invalid
        level = getattr(logging, level_name.upper(), logging.INFO)

        # how: prints to the console
        handler = logging.StreamHandler()
        handler.setLevel(level)
        # why: every line carries a timestamp/source/level, not just the
        # bare message
        log_format = "%(asctime)s %(name)s %(levelname)s %(message)s"
        formatter = logging.Formatter(log_format)
        handler.setFormatter(formatter)

        logger.addHandler(handler)
        # how: the logger's own level must be at least this low, or the
        # handler never even sees the message
        logger.setLevel(level)

    return logger
```

Also create empty `__init__.py` files in `src/documind/utils/` and `src/documind/models/`.

> Back to this step · Hint 1 · Hint 2 · Solution

Solution — state, nodes and graph in their own files, with a path log

Story — `models/llm.py`: one place builds the model. Graph nodes only receive the state, so each node calls `get_model()` instead of being handed a model. **If not:** every node would build its own `ChatOpenAI` with its own settings.

```
# src/documind/models/llm.py
from langchain_openai import ChatOpenAI

# why: one place to change the model for every node in the graph
MODEL_NAME = "gpt-4o-mini"

def get_model() -> ChatOpenAI:
    # why: graph nodes only receive the state, so each node builds its
    # model here instead of being handed one
    # how: ChatOpenAI reads OPENAI_API_KEY from the environment, which
    # load_config() filled from .env (and checked) at startup
    return ChatOpenAI(model=MODEL_NAME, temperature=0)
```

Story — `state.py`: the graph's whole public surface in one typed shape (Doc09). `path` uses `operator.add`, so each node *appends* its name — the graph keeps its own log of what ran. `start_state()` gives every field a known starting value. **If not:** field names would drift between nodes, and you'd need print statements to see the route taken.

```
# src/documind/state.py
import operator
from typing import Annotated, TypedDict

class DocState(TypedDict):
    # why: one typed shape listing everything any node can read or write
    task: str
    # how: operator.add appends each node's entries instead of replacing
    # the list — so path is a running log of the nodes that ran (Doc09)
    path: Annotated[list[str], operator.add]
    answer: str

def start_state(task: str) -> DocState:
    # why: every field starts with a known value, so no node ever has to
```

```
    # guess whether a key exists yet
    return {"task": task, "path": [], "answer": ""}
```

Story — `prompts/general.txt` + `utils/prompts.py`: the prompt text lives outside the code — Project 1's loader, with the folder changed to `src/documind/prompts`. **If not:** the wording would be buried in a node.

`src/documind/prompts/general.txt`

```
Answer this question from general knowledge, in at most 2 sentences.

Question: {task}
```

```
# src/documind/utils/prompts.py

# how: a path from the repo root — every command in this project is run
# from the repo root, the same rule the tests and scripts follow
PROMPTS_DIR = "src/documind/prompts"

def load_prompt(name: str) -> str:
    # why: prompts live in .txt files, not inside Python strings — you
    # can reword an agent's behavior without touching its code.
    # when: called whenever an agent needs its system prompt.
    # how: e.g. "worker" -> "src/documind/prompts/worker.txt"
    path = PROMPTS_DIR + "/" + name + ".txt"
    with open(path) as prompt_file:
        # how: strip() drops the blank line at the end of the file
        return prompt_file.read().strip()
```

Story — `nodes.py`: each node takes the state and returns only the fields it changes (Doc09's rule). `answer_node` is a Doc05 LCEL chain inside a node; it labels its answer as general knowledge, because later steps add answers from your documents. This file doesn't change again after this step. **If not:** a node returning the whole state could overwrite fields it never meant to touch.

```
# src/documind/nodes.py
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate

from documind.models.llm import get_model
from documind.state import DocState
from documind.utils.prompts import load_prompt

def clean_task(state: DocState) -> dict:
    # why: the first node — tidies the input once, so every later node
```

```
    # sees the same clean text
    return {"task": state["task"].strip(), "path": ["clean_task"]}

def answer_node(state: DocState) -> dict:
    # why: questions that aren't about the documents still get an answer,
    # clearly labelled as general knowledge, not as a document fact
    prompt = ChatPromptTemplate.from_template(load_prompt("general"))
    chain = prompt | get_model() | StrOutputParser()
    reply = chain.invoke({"task": state["task"]})
    answer = reply + "\n(general knowledge — not from your documents)"
    return {"answer": answer, "path": ["answer_node"]}
```

Story — `graph.py`: `build_graph()` wires nodes and edges and returns the compiled graph. Keeping it in a function means `main.py`, tests and scripts can all build the same graph. **If not:** the graph would exist only while one script runs top to bottom.

```
# src/documind/graph.py
from langgraph.graph import END, START, StateGraph

from documind.nodes import answer_node, clean_task
from documind.state import DocState

def build_graph():
    builder = StateGraph(DocState)
    builder.add_node("clean_task", clean_task)
    builder.add_node("answer_node", answer_node)

    # how: plain edges — START -> clean_task -> answer_node -> END
    builder.add_edge(START, "clean_task")
    builder.add_edge("clean_task", "answer_node")
    builder.add_edge("answer_node", END)
    return builder.compile()
```

Story — `main.py`: config first, then the graph, then a question loop that prints the path and the answer. **If not:** you couldn't see the graph working.

```
# src/documind/main.py
import sys

import openai

from documind.config import load_config
from documind.exceptions import MissingConfigError
from documind.graph import build_graph
from documind.state import start_state
```

```
from documind.utils.logger import get_logger

def main() -> None:
    # why: config first — a missing OPENAI_API_KEY stops the app here
    try:
        load_config()
    except MissingConfigError as e:
        print(f"Setup problem: {e}")
        sys.exit(1)

    logger = get_logger(__name__)
    graph = build_graph()
    print("DocuMind — ask a question, or /quit to exit.")

    while True:
        task = input("You: ").strip()
        if task == "/quit":
            break
        if task == "":
            continue

        try:
            result = graph.invoke(start_state(task))
        except openai.AuthenticationError:
            logger.error("Authentication failed — check OPENAI_API_KEY.")
            print("Login failed — check OPENAI_API_KEY in your .env file.")
            sys.exit(1)

        # why: path shows which nodes ran, in order — the graph's own log
        print(f"  path: {' -> '.join(result['path'])}")
        print(f"Answer: {result['answer']}")

if __name__ == "__main__":
    main()
```

Run it: `python -m documind.main`

```
DocuMind — ask a question, or /quit to exit.
You: What is LangGraph?
  path: clean_task -> answer_node
Answer: LangGraph is a library for building stateful, multi-step agent
workflows as graphs of nodes and edges.
(general knowledge — not from your documents)
You: /quit
```

(The answer is wrapped here to fit the page; the wording varies.)

Trace it by hand before moving on: after `clean_task`, the state is `{"task": "What is LangGraph?", "path": ["clean_task"], "answer": ""}`. After `answer_node`, `path` is `["clean_task", "answer_node"]` and `answer` holds the reply.